Retail AI AR Demos

Setup 1 — Design Document

All Models on Mobile Phone · Open-Source Deployable Stack

Project	Retail AI AR Demos
Scope	Setup 1: Stream + On-Device Inference
Status	DRAFT
Devices	RayNeo X3 Pro + Android Phone
Author	Zifeng Wang
Date	February 2026

Contents

Problem Statement

Background

Retail environments face two recurring challenges:

1. **Shoppers** need quick, accurate information about products — nutrition facts, allergens, price — without scanning barcodes or reading small-print labels.
2. **Store associates** spend significant time locating misplaced products and verifying shelf compliance against planograms, without any real-time guidance tool.

Augmented Reality (AR) glasses provide a hands-free, first-person view of the retail environment. Combined with an AI pipeline, they can surface contextual information directly in the associate's or shopper's field of view — or via voice — without interrupting the natural workflow.

Problem Definition

Core problem: How to run a complete AI pipeline (video ingestion, product detection, OCR, retrieval-augmented generation, speech I/O) on *commodity hardware* (AR glasses + Android phone) with *no dependency on cloud infrastructure or proprietary APIs*, at latency acceptable for real-time retail interaction.

Out of Scope (Setup 1)

- Running models on the glasses (Setup 2).
- Cloud-based LLM or cloud STT/TTS.
- Barcode scanning (may be added as Phase 3 enhancement).
- Multi-user sessions / back-office analytics dashboard.

Requirements

Functional Requirements

FR-1 · Video Streaming

- The AR glasses shall stream live camera feed to the phone over local WiFi (or phone hotspot) using MJPEG over HTTP.
- The phone shall display the incoming stream in real time.

FR-2 · Product Detection

- The phone shall run a real-time object detection model on each incoming frame to locate and classify retail products.
- Detection results (bounding boxes, class labels, confidence scores) shall be overlaid on the live stream view.

FR-3 · OCR on Product Labels

- Upon user trigger (tap or voice command), the phone shall run OCR on the region of interest (detection bounding box crop) to extract text: nutrition facts, ingredients, allergen warnings, price.
- Raw OCR text shall be passed to the RAG pipeline as additional context.

FR-4 · Speech-to-Text (STT)

- The phone shall capture voice input from its microphone.
- STT shall convert audio to text entirely on-device (no cloud API).
- Transcript shall be forwarded to the LLM/RAG pipeline.

FR-5 · Retrieval-Augmented Generation (RAG)

- The phone shall maintain a local knowledge base of retail products (nutrition, allergens, pricing) and shelf planograms.
- Given a user query + OCR context, the system shall retrieve the top-k relevant documents using embedding similarity.
- Retrieved context shall be injected into the LLM prompt.

FR-6 · On-Device LLM

- The phone shall run a quantized open-source LLM locally.
- The LLM shall answer user queries using only the retrieved context (no hallucination outside provided documents).
- Use-case 1 (Shopping Assist): answer questions about nutrition and allergens.
- Use-case 2 (Shelf Analysis): analyze detected products vs. planogram and generate associate guidance.

FR-7 · Text-to-Speech (TTS)

- The phone shall convert LLM output to speech and play it via the phone speaker or earphone.
- TTS shall run entirely on-device.

FR-8 · Shelf Analysis

- The system shall detect products on a shelf segment and compare their positions to a configurable planogram (zone → expected SKU mapping).
- Misplaced or missing items shall be identified and reported to the associate via voice and on-screen list.

Non-Functional Requirements

High-Level Approach & Detailed Design

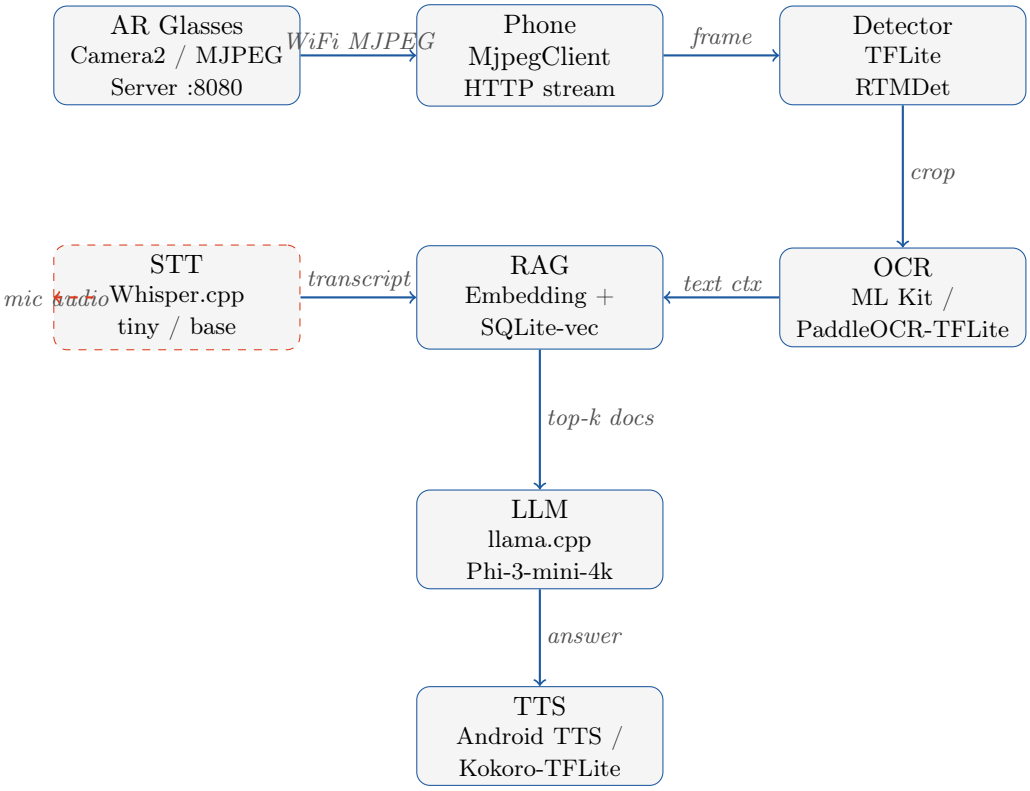
System Architecture Overview

Core principle: The AR glasses act purely as a *camera + display node*. The Android phone is the *inference hub* — every AI component runs on-device. No cloud service is required.

Data Flow Diagram (Setup 1):

ID	Requirement	Target
NFR-1	End-to-end voice latency	< 4 s from speech end to TTS playback
NFR-2	Detection frame rate	≥ 8 FPS on mid-range Android (Snapdragon 8 Gen 1+)
NFR-3	Network dependency	Zero cloud calls; local WiFi only (glasses $\leftrightarrow$ phone)
NFR-4	LLM model size	≤ 4 GB RAM footprint on phone
NFR-5	Offline operation	Full pipeline operational with no internet (phone hotspot OK)
NFR-6	Battery	≥ 2 h continuous operation on phone
NFR-7	Privacy	No user data leaves the local network
NFR-8	Open-source stack	All models/frameworks must be open-source and self-hostable

Table 1: Non-Functional Requirements



Component Design

Component 1 — Video Streaming

Component 2 — Product Detection

Component 3 — OCR

Component 4 — Speech-to-Text (STT)

Role	Glasses push live MJPEG to phone over WiFi
Glasses side	`MjpegServer` (existing): Camera2 API → JPEG frames → HTTP multipart stream on port 8080; dual-Surface for RayNeo left/right eye preview
Phone side	`MjpegClient` (existing): HTTP GET `/stream`, parse multipart, decode JPEG to `Bitmap`, deliver to pipeline
Frame throttle	Phone-side throttle: process at most 1 frame per 80 ms (≈12 FPS) for inference; display at stream rate
IP discovery	Glasses display own IP via `NetworkInterface` fallback; user enters IP once in phone app

Model	RTMDet-pico (fp16), `rtmdet_pico_fp16.tflite` (existing)
Framework	TensorFlow Lite + GPU delegate (fp16)
Input	640×640 px, BGR normalized
Output	Rotated bounding boxes (cx, cy, w, h, angle), class id, confidence
Post-process	NMS (IoU 0.45, score 0.30); rotated box drawing
Frame skip	`detectWithSkip`: atomic lock, skip frame if previous not done
Upgrade path	Swap to retail-trained model (see §4)

Trigger	User voice command "analyze this" or single tap
Input	Cropped Bitmap from highest-confidence detection box
Primary option	**ML Kit Text Recognition v2** (on-device, no network): Google's on-device OCR; Latin script + digits; returns `Text` blocks with bounding boxes
Fallback option	PaddleOCR TFLite export (see §4 for trade-off)
Output	Raw text string passed to RAG pipeline as `ocrContext`
Post-process	Simple keyword filter: extract lines containing allergen keywords (e.g. "nuts", "milk", "gluten") and nutrition facts pattern (`\d+\s*(g|mg|kcal)`)

Model	**Whisper.cpp** — `ggml-tiny.bin` (75 MB) or `ggml-base.bin` (142 MB)
Integration	Whisper.cpp Android JNI bindings or pre-built AAR
Trigger	"Ask" button or glasses temple single-click → start recording; double-click or button again → stop
Audio format	16 kHz, mono, 16-bit PCM; record with `AudioRecord`
Latency	tiny model: ≈ 0.5 s for 5 s audio on Snapdragon 8 Gen 1
Language	English (switchable via `language` param)

Component 5 — RAG (Retrieval-Augmented Generation)

Knowledge Base

Embedding Model

Data sources	(1) Product DB: name, SKU, brand, ingredients, nutrition facts, allergen flags; (2) Shelf planogram: zone id → expected SKU list, description
Format	JSON files bundled in `app/src/main/assets/`; loaded into SQLite on first run
Update path	Replace JSON files and rebuild, or REST pull from store backend (future)

Model	**all-MiniLM-L6-v2** converted to TFLite (22 MB); 384-dim embeddings
Indexing	At startup: embed each document chunk → store vector in SQLite (`sqlite-vec` extension) or plain `BLOB` column
Query embedding	At query time: embed (query + OCR text); cosine similarity against stored vectors
Top-k	Retrieve top-3 most relevant chunks; concatenate as context

Component 6 — On-Device LLM

Model	**Phi-3-mini-4k-instruct** (Q4_K_M quantized, ≈2.2 GB); strong instruction-following, fits most flagship phones
Framework	**llama.cpp** Android JNI or **MLC-LLM** Android AAR
Prompt template	`<|system|> You are a retail assistant. Answer ONLY from the provided context. If not in context, say "I don't have that information." \n<|context|> {top-k docs} \n<|ocr|> {ocr text} \n<|user|> {question}`
Context window	4096 tokens (Phi-3-mini-4k); sufficient for 3 product docs + OCR
Max new tokens	256 (Shopping assist: short answers; Shelf: short step-by-step)
Latency	≈2–3 s first-token on Snapdragon 8 Gen 2 with Q4

Component 7 — Text-to-Speech (TTS)

Primary	Android `TextToSpeech` API (system TTS engine, e.g. Google TTS pre-installed) — zero extra model, works offline if voice pack downloaded
Open-source fallback	**Kokoro** TFLite or **Coqui TTS** small model (e.g. `tts_models/en/ljspeech/glow-tts`) for fully self-contained TTS if system TTS is unavailable
Output	Phone speaker / Bluetooth earphone

Demo Flows

Demo A — Shopping Assist Agent

1. User wears glasses, looks at shelf/product.
2. Phone streams frames from glasses; RTMDet detects product.
3. User asks by voice: *"Does this contain nuts?"*
4. STT (Whisper.cpp) transcribes on-device.

5. User tap / voice trigger → OCR on detected product crop → extract ingredient/allergen text.
6. RAG: embed (query + OCR text) → retrieve product record from local DB.
7. LLM (Phi-3-mini): generate concise answer using retrieved context only.
8. TTS plays: *"Yes, this product contains tree nuts. Also contains milk."*
9. Answer text displayed on phone screen.

Demo B — Shelf Analysis

1. Associate wears glasses, scans shelf section.
2. RTMDet detects all visible products; positions mapped to shelf zones by Y-coordinate heuristic.
3. Compliance engine compares detected products per zone vs. planogram JSON.
4. Misplaced / missing items collected into structured list.
5. LLM generates associate guidance from planogram context + issue list.
6. TTS plays: *"Move the orange juice from zone 3 to zone 1, top shelf."*
7. Misplaced items listed on phone screen with zone instructions.

Module Map in Codebase

```
ProductARMobile/app/src/main/java/com/ultronai/productarmobile/
+-- ml/
|   +-- RTMDetDetector.kt          (existing - detection)
|   +-- OcrProcessor.kt            (NEW - ML Kit OCR wrapper)
|   \-- EmbeddingModel.kt          (NEW - all-MiniLM TFLite)
+-- llm/
|   \-- LlmClient.kt               (NEW - llama.cpp / MLC-LLM JNI)
+-- rag/
|   +-- KnowledgeBase.kt           (NEW - load product/planogram JSON)
|   +-- VectorStore.kt             (NEW - SQLite-vec retrieval)
|   \-- RagPipeline.kt             (NEW - orchestrate embed+retrieve)
+-- stt/
|   \-- WhisperStt.kt              (NEW - Whisper.cpp JNI)
+-- tts/
|   \-- TtsManager.kt              (NEW - Android TTS / Kokoro)
+-- shopping/
|   \-- ShoppingAssistAgent.kt     (NEW - orchestrate shopping flow)
+-- shelf/
|   +-- ZoneMapper.kt              (NEW - Y-coord $\rightarrow$ zone)
|   +-- ComplianceEngine.kt        (NEW - detected vs planogram)
|   \-- ShelfAnalysisAgent.kt      (NEW - orchestrate shelf flow)
\-- MainActivity.kt                (existing - extend with new UI)
```

Options & Trade-offs

LLM: llama.cpp vs. MLC-LLM vs. MediaPipe LLM

LLM Model Selection

OCR: ML Kit vs. PaddleOCR-TFLite

STT: Whisper.cpp vs. Android SpeechRecognizer

Option	Pros	Cons	Verdict
llama.cpp + JNI	Widest model support (GGUF); battle-tested; large community; easy quantization	JNI setup complexity; no native GPU (CPU/NNAPI only unless patched)	**Recommended (Phase 1)**
MLC-LLM AAR	OpenCL/Vulkan GPU acceleration; higher throughput; official Android AAR	Fewer supported models; larger AAR size; less mature for custom models	Good for Phase 2 / performance tuning
MediaPipe LLM Inference	Google-maintained; simple API; good GPU usage	Limited model selection; not all open models supported	Fallback if above fail
OpenAI-compat API	Zero phone RAM; best quality; fast prototyping	Requires network; not privacy-safe; cloud cost	Dev/debug only; not production

Table 2: LLM Option Comparison

Model (Q4_K_M)	Size	RAM	Notes
Phi-3-mini-4k (3.8B)	2.2 GB	2.6 GB	**Recommended** — strong reasoning, fits Snapdragon 8 Gen 1+
Gemma-2-2B-IT	1.5 GB	1.8 GB	Good quality; slightly less reasoning
Llama-3.2-1B-Instruct	0.7 GB	1.0 GB	Very fast; lower quality
Llama-3.2-3B-Instruct	1.9 GB	2.2 GB	Balance of speed and quality

Table 3: LLM Model Trade-off (all open-source, GGUF format)

Option	Pros	Cons	Verdict
ML Kit Text Recognition v2	Native Android; no model file needed; good accuracy on printed text; free offline mode	Google dependency; less control over model	**Use**
PaddleOCR TFLite export	Fully self-contained; excellent accuracy; Apache 2.0	Larger APK (+50–100 MB); integration effort	Phase 2 upgrade

Table 4: OCR Option Comparison

Option	Pros	Cons	Verdict
Whisper.cpp (tiny/base)	Fully offline; consistent accuracy; open-source (MIT)	JNI setup; 75–142 MB model file; no streaming	Use
Android SpeechRecognizer	Zero setup; streaming transcription; fast	Requires Google services + network; privacy concern	Dev/debug only

Table 5: STT Option Comparison

Embedding: all-MiniLM-L6-v2 vs. nomic-embed-text

Model	Pros	Cons	Verdict
all-MiniLM-L6-v2	22 MB; fast; well-supported TFLite export; 384-dim	Lower quality than larger models	Use
nomic-embed-text	Higher quality; Apache 2.0	Larger; TFLite port less tested	Phase 2

Table 6: Embedding Model Comparison

Detection Model: RTMDet vs. Retail-Trained YOLO

Model	Pros	Cons	Verdict
RTMDet-pico (existing)	Already integrated; fast; rotated boxes	General-purpose; not retail-trained	Phase 1
YOLOv8n / YOLOv11n fine-tuned on retail	Higher mAP on retail products; TFLite export available	Need labelled retail dataset; training time	Phase 2 target

Table 7: Detection Model Comparison

Rollout Plan & Risk Register

Rollout Plan

Risk Register

Phase	Name	Deliverables	Effort
P0	*Foundation stable*	MJPEG stream + RTMDet detection running stably on phone; frame throttle; IP display fix; TempleAction controls	1 week
P1	*Voice pipeline*	STT (Whisper.cpp JNI); Android TTS; "Ask" button in ProductARMobile; end-to-end voice echo test; RECORD_AUDIO permission	1–2 weeks
P2	*LLM on-device*	llama.cpp JNI integration; Phi-3-mini-4k GGUF loaded; hard-coded context Q&A test; latency benchmarks	2 weeks
P3	*OCR + basic Shopping Assist*	ML Kit OCR on detection crop; keyword extraction; LLM prompt with OCR context; Shopping Assist flow (voice → OCR → LLM → TTS)	1–2 weeks
P4	*RAG pipeline*	all-MiniLM TFLite embedding; SQLite-vec store; product/nutrition JSON KB; retrieval + LLM context injection; Shopping Assist with real product data	2 weeks
P5	*Shelf Analysis demo*	Planogram JSON; ZoneMapper (Y-heuristic); ComplianceEngine; ShelfAnalysisAgent; TTS guidance; on-screen misplaced list	1–2 weeks
P6	*Hardening & demo polish*	End-to-end latency tuning; UX improvements (glasses UI hints via stream back-channel); battery test; retail dataset for detection fine-tune	2 weeks

Table 8: Rollout Phases

#	Risk	Prob.	Impact	Mitigation
R1	**LLM OOM on phone** — Phi-3-mini exceeds available RAM on target device	Med	High	Profile RAM first; fall back to Llama-3.2-1B (0.7 GB); implement lazy load + unload
R2	**llama.cpp JNI integration complexity** — Build system issues, ABI mismatch	Med	Med	Use pre-built AAR from MLC-LLM or community Android builds; fallback to local HTTP server (Ollama) on phone

#	Risk	Prob.	Impact	Mitigation
R3	**LLM latency too high** — >5 s first-token unacceptable in demo	Med	Med	Switch to smaller model (1B or 2B); use MLC-LLM for GPU path; stream tokens to TTS
R4	**OCR accuracy on product labels** — Poor recognition on curved/reflective packaging	Med	Med	Pre-process crop (contrast, grayscale, resize); try PaddleOCR TFLite; fall back to barcode lookup
R5	**WiFi instability** — Stream drops, glasses and phone lose connection	Low	High	Auto-reconnect in MjpegClient; increase socket timeout; prefer phone hotspot over shared WiFi
R6	**Detection model not retail-ready** — RTMDet misses or misclassifies retail products	High	Med	For P0–P3: use general model + OCR as fallback; P6: fine-tune YOLOv8n on retail dataset (Roboflow)
R7	**Whisper.cpp JNI build on ARM64** — NDK/CMake config issues	Med	Med	Use community Android Whisper AAR (whispercpp-android); fallback to Android SpeechRecognizer in dev
R8	**Battery drain** — Running stream + detector + LLM > 2 h	Med	Med	Throttle detection to 8 FPS; load LLM only on demand; unload after 30 s idle
R9	**Planogram data not available** — No real store planogram for Shelf demo	Med	Med	Build synthetic planogram JSON for demo environment; demo on a manually arranged shelf
R10	**RayNeo dual-Surface camera not working** — Single Surface causes blank or frozen preview	Med	Med	Port Mahjong app dual-Surface startup logic (wait `surfaceList.size==2` before openCamera)

Success Metrics

Open-Source Model Inventory

Phone Hardware Targets

Metric	Target
End-to-end voice latency	< 4 s (speech end $\rightarrow$ TTS start)
Shopping Q&A accuracy	Correct allergen / nutrition answer for $\geq 80\%$ of test queries
Shelf compliance detection	Correctly identify misplaced item in ≥ 3 of 4 test scenarios
Continuous operation	≥ 2 h without crash or OOM on target phone
Stream frame rate	≥ 8 FPS detection throughput at steady state
Zero cloud dependency	Pipeline fully functional with no internet connection

Table 10: Demo Success Metrics

Component	Model	Framework	License	Source
Detection	RTMDet-pico	TFLite	Apache 2.0	OpenMMLab
Detection (P6)	YOLOv8n fine-tuned	TFLite	AGPL-3.0	Ultralytics
OCR	ML Kit v2	Native Android	Proprietary-free use	Google
OCR (P2)	PaddleOCR	TFLite	Apache 2.0	PaddlePaddle
Embedding	all-MiniLM-L6-v2	TFLite	Apache 2.0	sentence-transformers
LLM	Phi-3-mini-4k	GGUF / llama.cpp	MIT	Microsoft
LLM alt	Llama-3.2-3B-Instruct	GGUF / llama.cpp	Llama 3.2 License	Meta
STT	Whisper tiny/base	Whisper.cpp	MIT	OpenAI
TTS	Android TTS	System API	Proprietary-free use	Google
TTS (fallback)	Kokoro	TFLite	Apache 2.0	hexgrad
LLM runtime	llama.cpp	JNI	MIT	ggerganov
LLM runtime alt	MLC-LLM	AAR	Apache 2.0	mlc-ai
Vector store	sqlite-vec	Extension	Apache 2.0	asg017

Table 11: Full Open-Source Model & Library Inventory

Device tier	**Example**	**Chip**	**Expected LLM perf.**
High (primary target)	Samsung S23 / Pixel 8 Pro	Snapdragon 8 Gen 2 / Tensor G3	Phi-3-mini Q4: ≈2–3 s TTFT
Mid	Samsung A54 / Pixel 7a	Exynos 1380 / Tensor G2	Use Llama-3.2-1B for $<$ 4 s TTFT
Low	Snapdragon 778G class	SD 778G	Llama-3.2-1B only; detection at 6 FPS

Table 12: Target Android Hardware